

In the name of ALLAH



**Amirkabir University of Technology
(Tehran Polytechnic)**

Deep Neural Network Course By Dr. Safabakhsh
spring 2025

Assignment (03)

Mohammad Sadegh Eftekhari

403131014

Sadegh.eft@aut.ac.ir

Contents

IMPORTS	2
The DATA	3
Preprocessing	3
Visualizations.....	6
Q1: Reduce dimensions to 8, 4, and 2. Apply a classifier to each and compare results.....	7
Q2: Plot the distribution of the data in the dataset using a self-organizing network on a 2D map.	10
Q3: Changing Key Parameters and analyze the change in performance	11
Q5: Compare the SOM to PCA or/and t-SNE	18

IMPORTS

1. `ucimlrepo.fetch_ucirepo`
For fetching the Bank Marketing dataset directly from the UCI Machine Learning Repository.
2. `pandas`
For manipulating DataFrames, handling missing values, encoding categorical features.
3. `matplotlib.pyplot`
For creating various plots such as histograms, scatter plots, and heatmaps.
4. `numpy`
For numerical operations, array manipulation, random number generation, and mathematical functions.
5. `seaborn`
For generating enhanced visualizations, specifically used here for plotting the heatmap.
6. `warnings`
For suppressing warning messages that might appear during code execution.
7. `sklearn.preprocessing.MinMaxScaler`
For scaling the numerical features to a specific range.
8. `sklearn.decomposition.PCA`
For performing Principal Component Analysis, used here for dimensionality reduction, to visualize higher-dimensional SOM outputs in 2D and as a benchmark comparison method.
9. `sklearn.manifold.TSNE`
For performing t-Distributed Stochastic Neighbor Embedding, a non-linear dimensionality reduction technique used for visualizing high-dimensional data.
10. `sklearn.metrics.adjusted_rand_score, normalized_mutual_info_score`
For evaluating the quality of the SOM by comparing the resulting clusters against the true class labels.
11. `sklearn.ensemble.RandomForestClassifier`
For implementing the classification model used to evaluate the effectiveness of the original features versus the features derived from dimensionality reduction techniques (SOM, PCA, t-SNE).
12. `sklearn.model_selection.train_test_split`
For splitting the dataset into training and testing sets.

13. sklearn.metrics.accuracy_score, classification_report

For evaluating the performance of the RandomForestClassifier, providing metrics like overall accuracy and detailed precision, recall, and F1-scores for each class.

The DATA

To load the data, we are provided a link in the assignment instruction which contains a piece of code to download and import the data directly into the python code. So the code used to load the data, is just the provided code in the 'UCI' website

Preprocessing

First step of the preprocessing is the filling of the missing values. Checking the missing values using the 'isnull' function we get the following results:

	Total	Percent
poutcome	36959	0.8175
contact	13020	0.2880
education	1857	0.0411
job	288	0.0064
age	0	0.0000
marital	0	0.0000
default	0	0.0000
balance	0	0.0000
housing	0	0.0000
loan	0	0.0000
day_of_week	0	0.0000
month	0	0.0000
duration	0	0.0000
campaign	0	0.0000
pdays	0	0.0000
previous	0	0.0000

As we can see, only 4 features contain null values.

1. **poutcome:** missing so many values, so the best option is to just treat the missing as a category (we just fill them with 'unknown')

2. **contact:** for this one we only have 2 unique values ('cellular' and 'telephone'), we can't really replace the missing with any of them sense theyre missing close to a third of the values, so we just fill it with 'unknown'
3. **education:** we're gonig to fill the data with the most frequent value, becuae we're only missing about 5% of them, and also even in the real world, for the education feature of actual people this would probably work the best out of all the methods we can impelement.
4. **job:** the missing percentage is too small to have an impact on the performnace of our model, so we're going to do the same thing as the 'education'.

now if we take a look at the missing value count again, we would see this:

	Total	Percent
age	0	0.0
job	0	0.0
marital	0	0.0
education	0	0.0
default	0	0.0
balance	0	0.0
housing	0	0.0
loan	0	0.0
contact	0	0.0
day_of_week	0	0.0
month	0	0.0
duration	0	0.0
campaign	0	0.0
pdays	0	0.0
previous	0	0.0
poutcome	0	0.0

Using the 'dtypes' function, we can see what type of data each column contains. The labels are just 'yes' or 'no' which have to be converted into '1' and '0'. But the features contain 4 types of entries:

1. Numerical
2. Binary
3. Ordinal categorical
4. Normal categorical

We obviously don't want to perform anything on the numerical ones, but moving to the binary ones:

1. loan
2. housing
3. default

they can just get replace with '1' and '0', so we just use the 'map' function to map 'yes' to '1' and 'no' to '0'.

we have 2 features that are not binary, but are ordinal, which means we can benefit from assigning each value a number, and letting the network think the order of numbers matter:

1. **education**: the greater the education level, the greater the number we assign them
2. **month**: you know how it goes

we again use the 'map' function, to map each unique value to a digit, but we do it based on the order this time, like the number increases as the education lever does.

ok now we should have only '4' features with missing values, lest see what they have as values:

```
unique values of job: ['management' 'technician' 'entrepreneur' 'blue-collar' 'retired' 'admin.'  
 'services' 'self-employed' 'unemployed' 'housemaid' 'student']  
unique values of marital: ['married' 'single' 'divorced']  
unique values of contact: ['unknown' 'cellular' 'telephone']  
unique values of poutcome: ['unknown' 'failure' 'other' 'success']
```

These features are neither 'binary' nor 'ordinal' so we're just going to use the one-hot encoding, we use the 'get_dummies' function for this purpose. It should be noted, that our dataset now has a lot more features, sense we used the 'one-hot' encoding to convert categorical features into numerical ones

Now when we check to see how many categorical features are left, we get hit with an empty list

```
[]
```

Next step on the 'preprocessing' list is handling duplicate data, we use the 'duplicated' function with the 'sum' to check how many rows are duplicates, this is the output we got:

```
0
```

After all of these, the only thing is, now converting the 'Boolean' data into int (because we can't really use the Boolean type in the future tasks) which we do using the 'astype' function.

So now both the features and the labels are numerical, and we can move one to the actual tasks, provided in the assignment instruction, I did use 'minmaxscaler' as the one last bit of preprocessing though :)

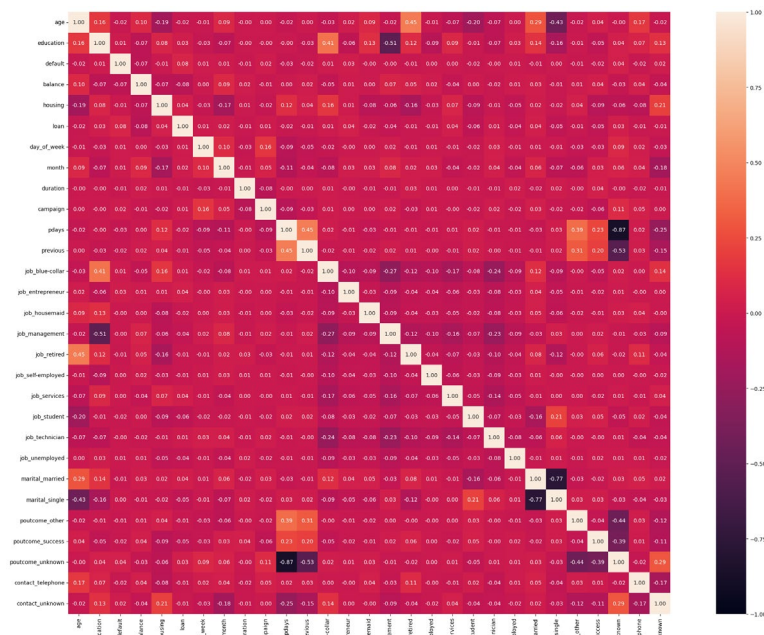
Visualizations

I did visualize 2 things, the 1st is the distributions of each feature and the 2nd is the correlation between them.

For the first one, we just use the 'hist' function provided by the 'matplotlib' library.



And for the 2nd one we use the 'heatmap' function, provided by the 'seaborn' library.



Looking at the 'distributions' what caught my eye, was the fact that the feature 'day_of_week' contains numbers from '1' to '30' which means, it probably should have been 'day_of_month' and not week, but I'm not going to touch it. We also can see that some features, like the 'balance', 'duration', ect. Are pretty skewed.

the 'heatmap' is pretty straight forward, and doesn't really need any analyzing, like of course the married and not-married features are going to have a strong correlation.

Q1: Reduce dimensions to 8, 4, and 2. Apply a classifier to each and compare results.

We're looking to see how reducing dimensions affects how well we can classify things. To do this, we use a Self-Organizing Map (SOM) to shrink the feature space down to 8D, 4D, and 2D. Then, we check how a Random Forest classifier does with these smaller datasets compared to the original, full-dimensional data.

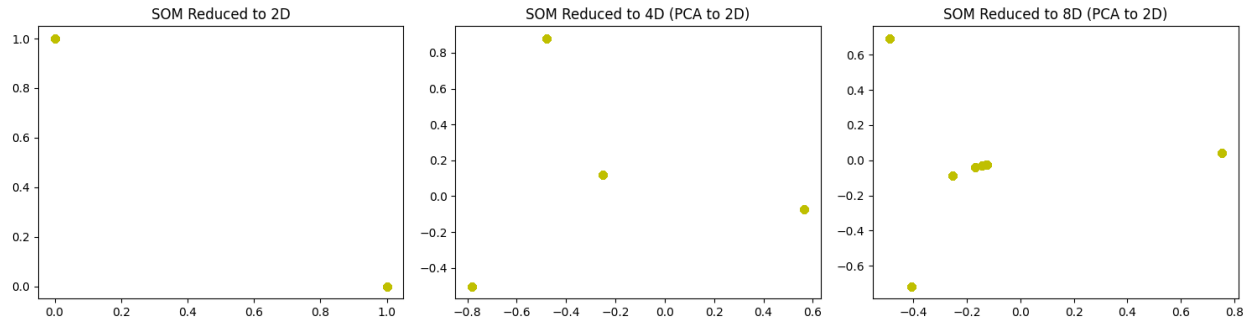
To implement the 'SelfOrganizingMap' class I defined the following methods:

1. **Initializer:** Like every other class, the first step would be to define an 'initializer' which initializes the following variables:
 - **input_dim:** The number of features in the input data (i.e., dimensionality of each data vector).
 - **map_dim:** Shape of the SOM grid (e.g., (2, 4) creates 8 nodes). Can be 1D (e.g., (10, 1)) or 2D.
 - **learning_rate:** Initial learning rate for weight updates. Decays over time.
 - **sigma:** Controls the neighborhood radius around the BMU; if not set, defaults to half the max grid dimension.
 - **weights:** A 3D array of shape (grid_x, grid_y, input_dim), initialized randomly. Each SOM node has a weight vector of length input_dim.
2. **Best Matching Unit (BMU):** as the name this method Finds the Best Matching Unit (BMU) for an input vector 'X'.
It Computes the Euclidean distance from 'X' to every node's weight vector. And then returns the (i, j) coordinates of the closest matching node (smallest distance).
3. **Update weights:** Updates the weights of the BMU and its neighbors using a Gaussian neighborhood function.
 - Applies exponential decay to the learning rate and sigma (radius) as training progresses.
 - Builds a coordinate grid for the SOM.
 - Computes squared distances from all nodes to the BMU.
 - Uses a Gaussian to define how much each node is influenced based on its distance to the BMU.
 - Updates weights with: $w_{ij} \leftarrow w_{ij} + influence \cdot (x - w_{ij})$

4. **Fit:** trains SOM on dataset 'X'. Iterates over the dataset 'num_iterations' times. And for each sample in 'X', it finds the BMU and updates the weights of the SOM accordingly.
5. **Transform:** Converts each input vector into a one-hot encoded representation of its BMU.
 - Finds the BMU coordinates for each input.
 - Converts BMU position into a flat index in the grid.
 - Returns a matrix of shape (N, 'num_nodes') where each row is a one-hot vector.
6. **Getting BMU indices:** Returns the raw (i, j) BMU coordinates for each input vector. Useful for visualization.

We're passed the hard part which is implementing what we're going to use for the rest of the assignment, now we have to do the next part of the first task, which is use the implemented class to for dimensionality reduction. For this purpose, we just call the class and pass the desired map dimension, and then we call the 'fit' method. After that we call the 'transform' method and that's it.

I then wanted to visualize the results for myself, **so this part is not wanted in the assignment**, but I did it anyways, to be able to visualize the 4D and 8D representations, we need to first bring them to 2D. I did this using the 'PCA' function, from the 'sklearn' library. And here is the result of it.



It looks interesting, it doesn't really mean that much, except the fact that we can see, more dimensions capture more details, and that's it.

The next steps of this task, are pretty straight forward, we now have 4 representations of the same data, one without any dimensionality reduction, one 8D, one 4D and another 2D representation. We now split all of them and use a classifier to classify and evaluate the performance on each of them.

I used the "RandomForestClassifier" from the 'sklearn' library. The results are shown here:

Original Features:

Accuracy: 0.9011390025434037

SOM 8D:

Accuracy: 0.8828928452947031

SOM 4D:

Accuracy: 0.8852150834899922

SOM 2D:

Accuracy: 0.8793541966161672

I also used the “classification_report” function from the ‘sklearn’ library for a better result representer:

Original Features:

Metric	Class 0	Class 1	Accuracy	Macro Avg	Weighted Avg
Precision	0.92	0.65	0.90	0.79	0.89
Recall	0.97	0.39		0.68	0.90
F1-score	0.95	0.49		0.72	0.89
Support	7952	1091	9043	9043	9043

SOM 8D:

Metric	Class 0	Class 1	Accuracy	Macro Avg	Weighted Avg
Precision	0.88	0.00	0.88	0.44	0.77
Recall	1.00	0.00		0.50	0.88
F1-score	0.94	0.00		0.47	0.82
Support	7952	1091	9043	9043	9043

SOM 4D:

Metric	Class 0	Class 1	Accuracy	Macro Avg	Weighted Avg
Precision	0.89	0.63	0.89	0.76	0.86
Recall	0.99	0.12		0.55	0.89
F1-score	0.94	0.20		0.57	0.85
Support	7952	1091	9043	9043	9043

SOM 2D:

Metric	Class 0	Class 1	Accuracy	Macro Avg	Weighted Avg
Precision	0.89	0.61	0.88	0.75	0.85
Recall	0.99	0.08		0.54	0.88
F1-score	0.94	0.14		0.54	0.84
Support	7952	1091	9043	9043	9043

The Original Features model performs best overall, achieving the highest F1-score for class 1 (0.49), which indicates better balance between precision and recall for the minority class. As dimensionality is reduced using SOM (from 8D to 2D), performance on class 1 significantly declines—particularly in SOM 8D, where class 1 is not detected at all (F1-score = 0.00). SOM 4D and SOM 2D slightly improve class 1 precision but suffer from very low recall, suggesting the model struggles to identify positive cases as dimensionality decreases.

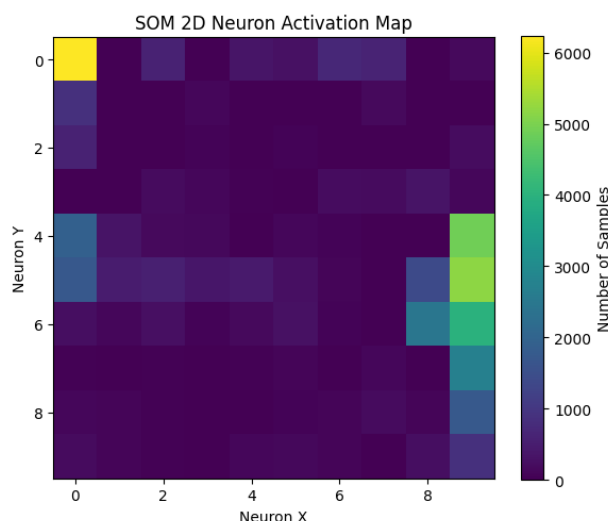
Q2: Plot the distribution of the data in the dataset using a self-organizing network on a 2D map.

First step of this task, is to train the SOM again. So, we call the class and pass the following values:

- **input_dim=X.shape[1]:** The input dimension matches your data feature size.
- **map_dim=(10, 10):** You create a 10×10 grid (100 neurons total).

We then train the SOM on the dataset 'X'. next step is getting the BMUs, which is done just by calling the method designed for the task (which we explained before). And then just counting the samples per neuron. For each sample's BMU (i,j), you increment the corresponding cell in the heatmap.

We then use the 'heatmap' function of the 'seaborn' library to simply visualize the distribution.



After training a 10×10 Self-Organizing Map on the dataset, a 2D activation heatmap was plotted to visualize the distribution of the data samples across the SOM neurons. The resulting heatmap reveals that the dataset is highly concentrated in specific regions, particularly at the neuron located at (0,0) and the neurons along the rightmost column (especially around (9,4) to (9,7)). These neurons showed very high activation values, with the top neuron mapping over 6000 samples, indicating significant clustering in the original data space. In contrast, large portions of the SOM map remained sparsely activated, reflecting areas with few or no data points. Overall, the SOM effectively preserved the topological structure of the dataset, successfully representing dense and sparse regions within the 2D neuron grid.

Q3: Changing Key Parameters and analyze the change in performance

This task is asking us to find the optimal value for each parameter, which are the 'Grid Size', the 'Learning rate' and the 'number of iterations'. The following table shows what values for each parameter were used.

Parameter	Values You Used
Grid Sizes	(5×5), (10×10), (15×15)
Learning Rates	0.1, 0.5, 0.9
Iterations	50, 100

We systematically trained SOMs under different configurations and measured the resulting quality of data mapping using PCA explained variance.

Because this task was taking too long to run, and while running my pc either crashed, or the power went off and I lost progress, I decided to keep the code for running each 'grid size' separated. To do this I decided to make the grid search a function that I can call for each 'grid size'.

This is how the function works:

- Loops over learning rates and number of iterations (2 hyperparameters).
- For each (lr, iter) combination:
 - Trains a Self-Organizing Map (som.fit(X)).
 - Transforms the data into the SOM representation.
 - Applies PCA to reduce the SOM output into 2D for plotting.
 - Calculates the total explained variance of the PCA (how much of the structure is preserved in 2D).
 - Stores the parameters and explained variance into a results list.

The function then returns a list of dictionaries with results.

I then did the same thing for the visualization, I turned my previous code into a function, so I can visualize the results for each 'grid size' separately.

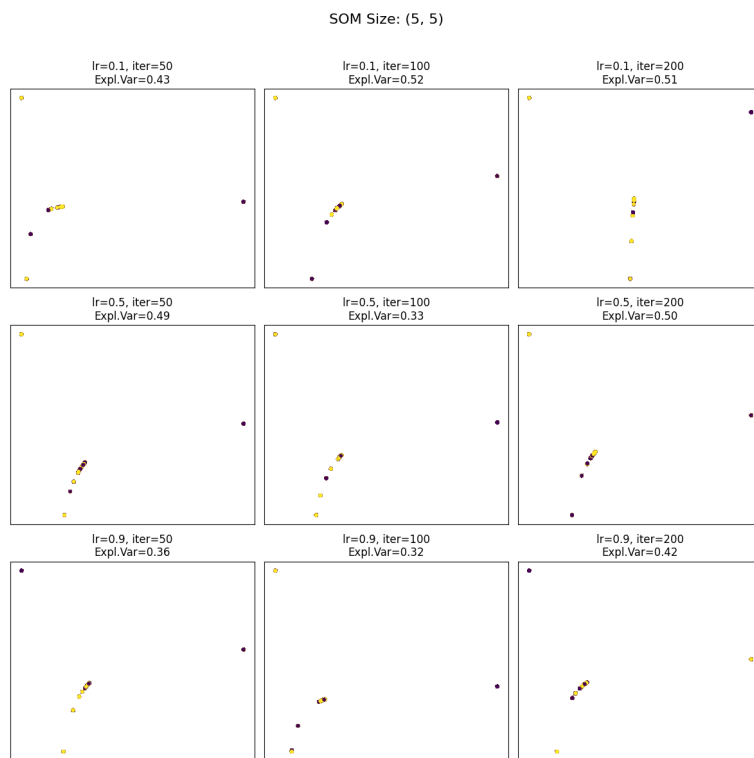
the input, is just the output of the previous function, which were the results.

Inside the function:

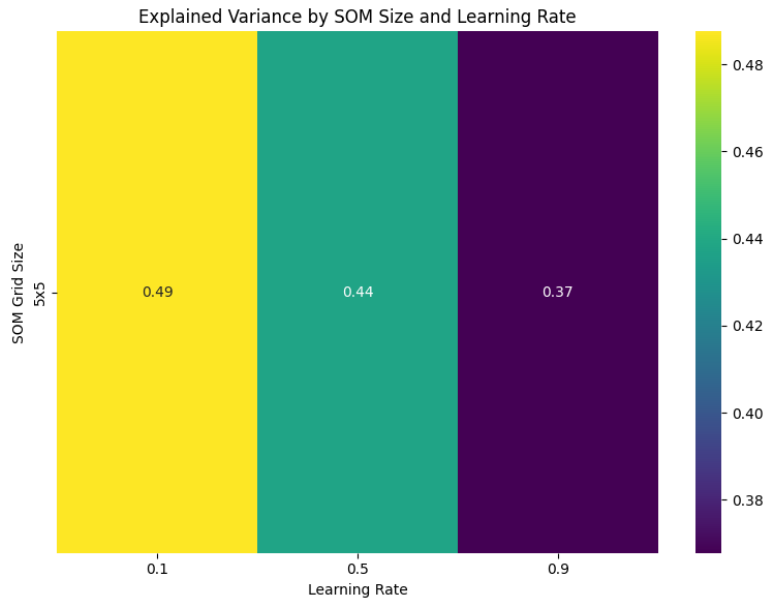
- Converts the results list to a Pandas DataFrame.
- Prepares a pivot table: rows = SOM size, columns = learning rate, values = explained variance.
- Plots a heatmap to visualize how the explained variance changes based on different values.

We then call each function separately for each 'grid size'. And here are the results for each of them:

Grid Size: (5, 5): 205m to run



Learning Rate	Iterations	Explained Variance
0.1	50	0.4343
0.1	100	0.5225
0.1	200	0.5058
0.5	50	0.4873
0.5	100	0.3282
0.5	200	0.4981
0.9	50	0.3647
0.9	100	0.3207
0.9	200	0.4180

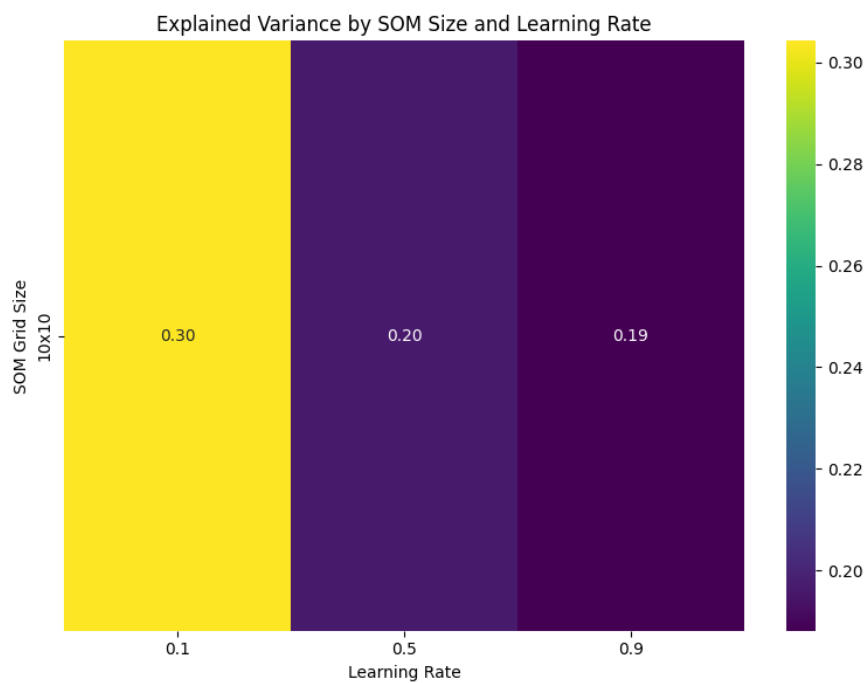


Highest explained variance overall (peak at 0.5225 with learning rate 0.1, 100 iterations). Performs most consistently across all learning rates. Learning rate 0.1 generally outperforms higher rates. Increasing iterations from 50 to 100 improves variance, but gains plateau or slightly drop at 200.

Grid Size: (10, 10): 253m to run

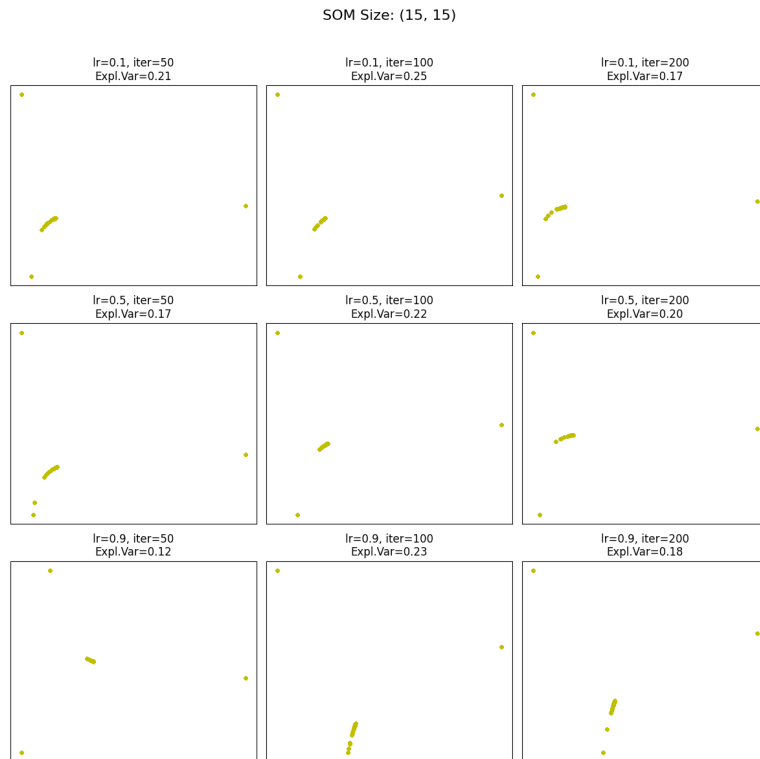


Learning Rate	Iterations	Explained Variance
0.1	50	0.3240
0.1	100	0.1742
0.1	200	0.4146
0.5	50	0.1349
0.5	100	0.2593
0.5	200	0.1934
0.9	50	0.1696
0.9	100	0.1862
0.9	200	0.2086

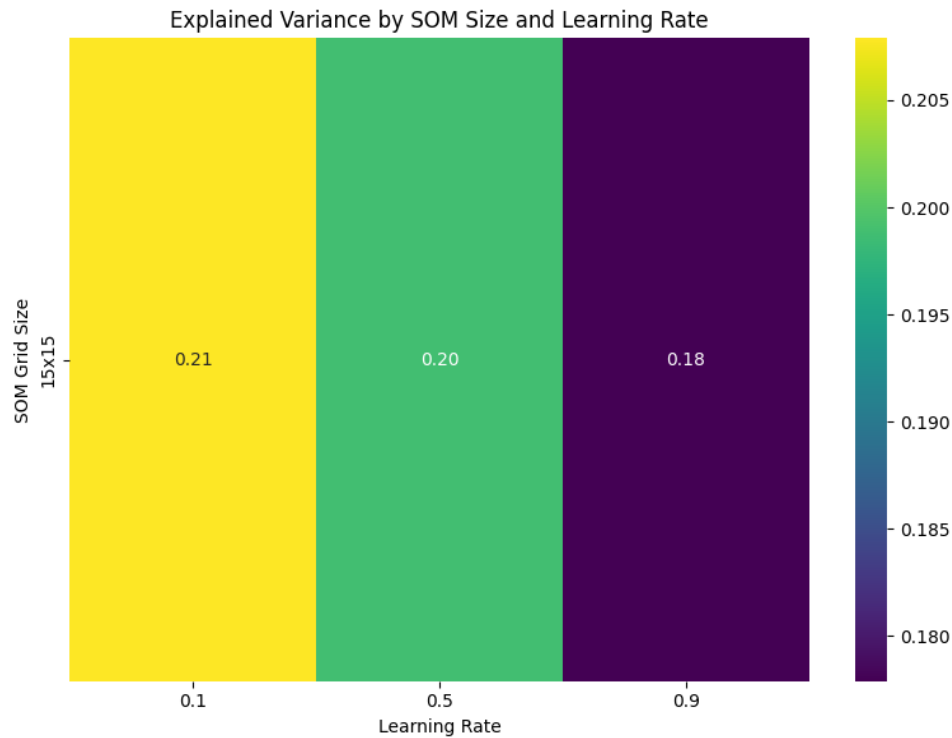


More inconsistent performance; best variance is 0.4146 (learning rate 0.1, 200 iterations). Variance fluctuates more with different hyperparameters. Learning rate 0.1 still performs best, but less reliably than with 5x5.

Grid Size: (10, 10): 350m to run



Learning Rate	Iterations	Explained Variance
0.1	50	0.2067
0.1	100	0.2518
0.1	200	0.1652
0.5	50	0.1706
0.5	100	0.2213
0.5	200	0.2044
0.9	50	0.1207
0.9	100	0.2287
0.9	200	0.1842



Shows the lowest overall explained variance. Peaks at 0.2518 (learning rate 0.1, 100 iterations). High learning rates (especially 0.9) generally underperform. Performance appears to degrade as grid size increases, possibly due to over-fragmentation or loss of generalization.

So the results show the following points:

- Smaller SOM grids (like 5×5) tend to capture more meaningful structure in the data, reflected in higher explained variance.
- Learning rate '0.1' consistently yields the best results across all grid sizes.
- 100 iterations is often a sweet spot (more may not help and can sometimes hurt performance).
- As grid size increases, explained variance generally decreases, suggesting diminishing returns or possible overfitting/noise amplification.

Q4: Cluster data using a self-organizing map and evaluate cluster quality using account status labels.

In this task, a Self-Organizing Map (SOM) was used to perform unsupervised clustering on the dataset without using the true labels during training. The SOM was trained with an input dimensionality equal to the number of features, a map size of 10×10 neurons, a learning rate of 0.5, and 100 iterations.

After training, each data point was assigned to a cluster based on the location of its Best Matching Unit (BMU) on the SOM grid. The two-dimensional BMU coordinates were converted into unique cluster labels by using the formula $\text{cluster_label} = \text{row} \times \text{width} + \text{column}$, ensuring that each neuron corresponds to a unique cluster identifier.

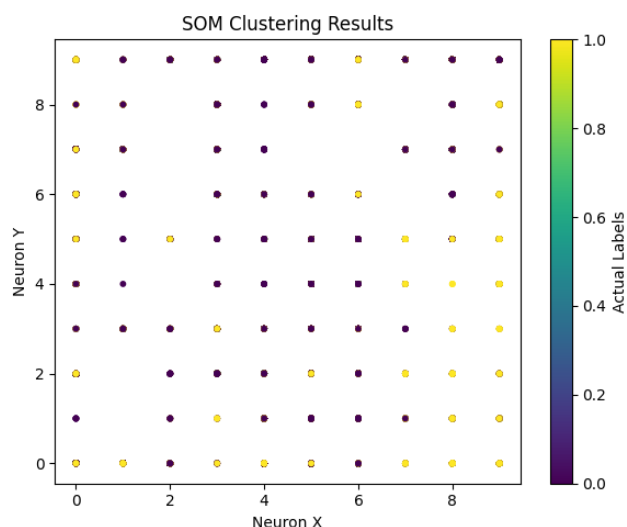
To evaluate the clustering performance, the predicted cluster labels were compared with the actual labels (whether an account was opened or not) using two metrics:

- **Adjusted Rand Index (ARI):** Measures the similarity between the clustering and the ground truth, adjusting for chance. ARI values close to 1 indicate highly accurate clustering.
- **Normalized Mutual Information (NMI):** Measures the mutual information between the cluster assignments and the true labels, normalized to be between 0 (no mutual information) and 1 (perfect correlation).

The resulting ARI and NMI scores were reported. Additionally, a scatter plot was generated, where each point represents a data sample's BMU location on the SOM grid, colored according to the actual labels. This visualization provides insight into how well the SOM clusters correspond to the true class distribution.

And this is the output for the 4th task of this assignment:

```
Adjusted Rand Index (ARI): 0.0100
Normalized Mutual Information (NMI): 0.0215
```



These values are very close to zero, meaning the clusters formed by the SOM do not align well with the actual labels (account opened vs not opened).

The SOM grouped data based on feature similarity, but this similarity does not reflect the true label structure in this case.

Q5: Compare the SOM to PCA or/and t-SNE

In this task, the performance of the Self-Organizing Map (SOM) was compared with two popular dimensionality reduction techniques: Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE).

First, PCA was applied to the dataset to reduce its dimensionality to 8 components. Separately, t-SNE was applied to reduce the data directly to 2 components, which is typical given t-SNE's nature as a visualization-focused algorithm.

The resulting representations from PCA and t-SNE were each split into training and testing subsets using an 80/20 train-test split. A Random Forest Classifier was then trained separately on the PCA-reduced and t-SNE-reduced feature sets. The classification performance was evaluated using accuracy on the test set.

The steps followed were:

- Apply PCA (8 components) and t-SNE (2 components) to the dataset.
- Split the reduced data into training and testing subsets.
- Train a Random Forest Classifier on each reduced dataset.
- Predict and calculate the classification accuracy on the test sets.

The final reported metrics were the classification accuracies obtained from PCA and t-SNE transformations. These results provide a basis for comparing how well the reduced representations capture information relevant to classification, offering an indirect comparison to the SOM's representation quality.

And here are the outputs:

```
PCA Classification Accuracy: 0.8803494415570055  
t-SNE Classification Accuracy: 0.8708393232334403
```

The classification results show that PCA achieved a slightly higher accuracy (88.03%) compared to t-SNE (87.08%) when used as input features for a Random Forest classifier. This suggests that PCA, despite being a linear method, was effective at preserving discriminative features in the data for this specific classification task.

On the other hand, t-SNE is primarily designed for visualization, not classification, and typically reduces data to 2 or 3 dimensions to reveal clusters rather than preserve global structure. Its slightly lower accuracy is expected and acceptable in that context.

Overall, PCA proved more suitable for downstream predictive modeling, while t-SNE remains valuable for gaining intuitive insights into data structure and separability in 2D.